

STREAM PROCESSING

Le Ngoc Thanh – Nguyen Ngoc Thao
{lnthanh, nnthao}@fit.hcmus.edu.vn

Outline

- Stream processing fundamentals
- Streaming API in Spark

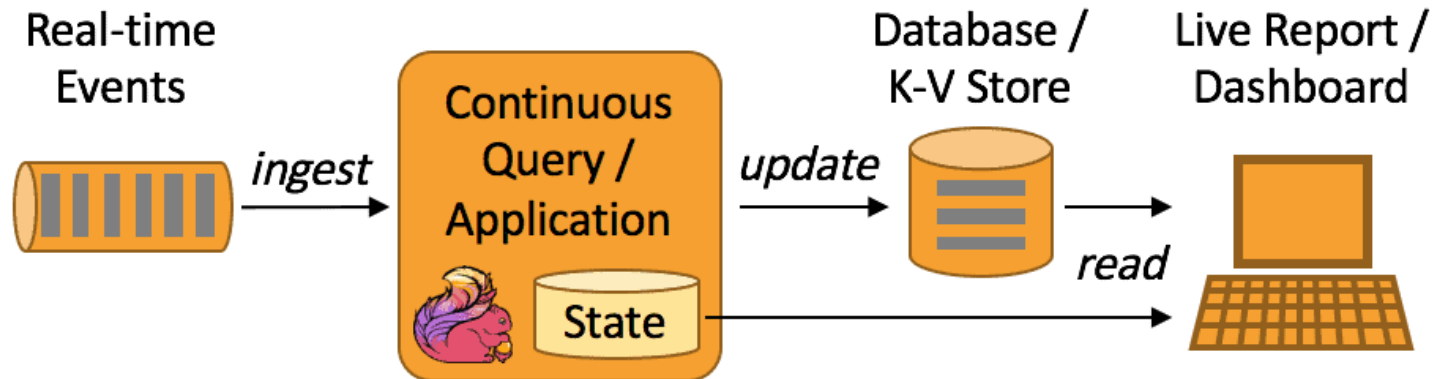


Stream processing



Stream processing

- **Stream processing** is the act of **continuously incorporating new data** to compute a result.



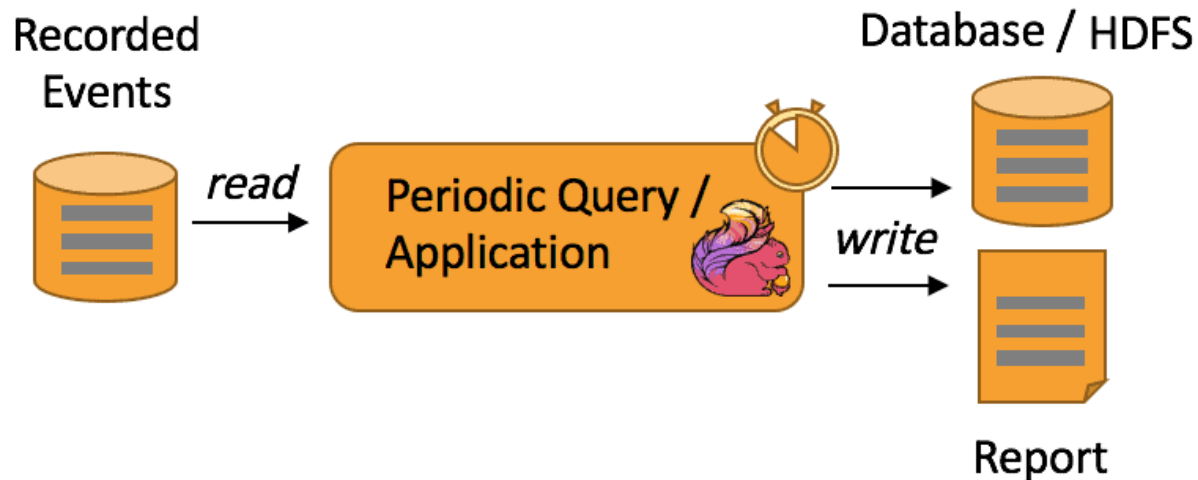
- User applications can compute different queries over the stream of events.
 - E.g., tracking a running count of each type of event or aggregating them into hourly windows.

Stream processing

- The **input data** has **no predetermined beginning or end**.
 - It simply forms a series of events (e.g., credit card transactions, clicks on a website, or sensor readings from IoT devices).
- The application will output multiple versions of the result (**output data**) as it runs.
- Or perhaps **keep it up to date** in an external “sink” system.

Batch processing

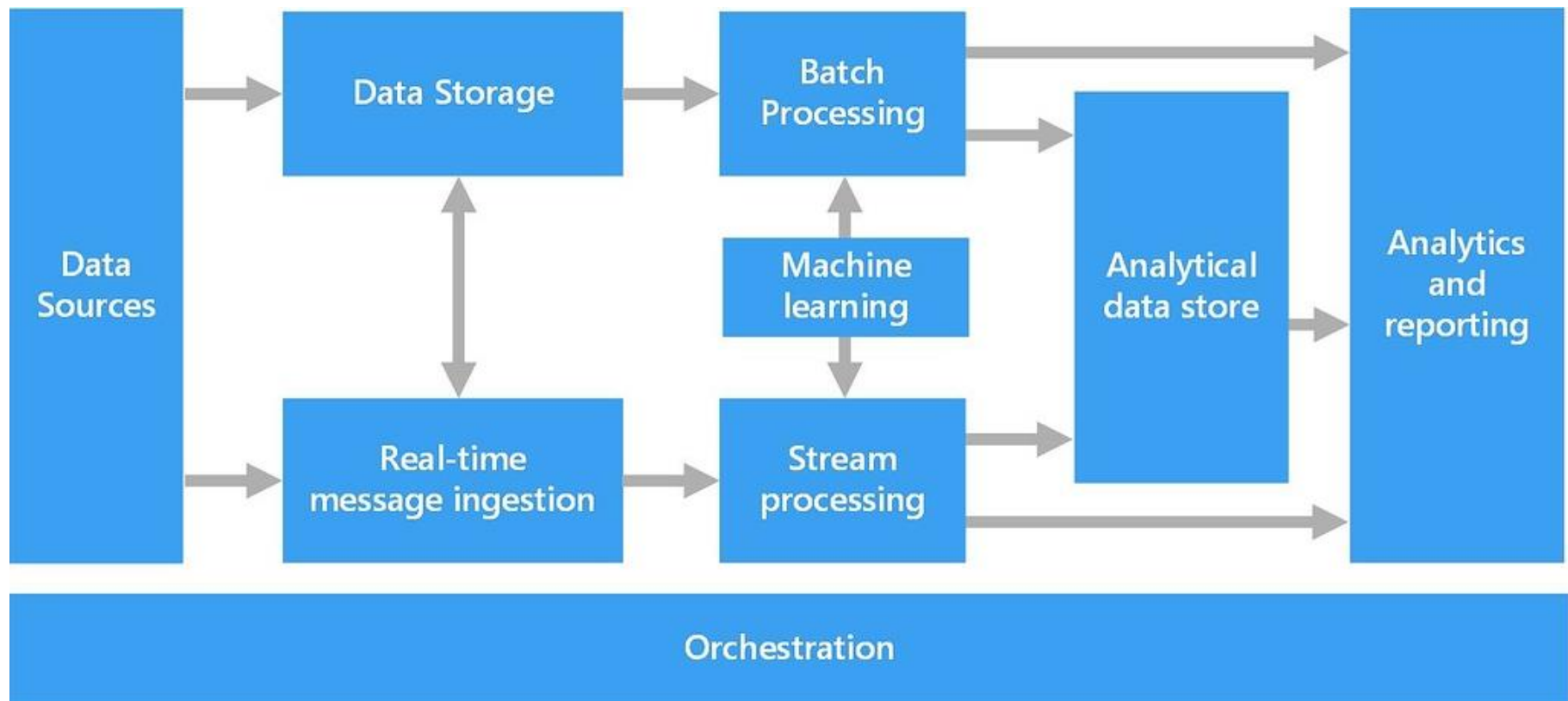
- **Batch processing** runs the **computation on a fixed dataset**.
 - The data is often large-scale and stored in a data warehouse
 - It contains all the historical events from an application, e.g., all website visits or sensor readings for the past month



- It also takes a query to compute, like stream processing, but **only computes the result once**.

Stream and Batch processing

- These processing modes are different, yet they often need to work together in practice.



Stream processing: Use cases



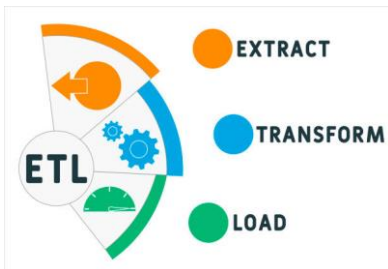
Notifications and alerting

- A notification or alert should be triggered if some sort of event or series of events occurs
- E.g., credit card fraud detection, elderly home monitoring



Real-time reporting

- Dashboards are common in several organizations to announce live updates to audiences.
- E.g., live reports for stock market center, city traffic, etc.



Incremental ETL

- We can incorporate new data within seconds, enabling users to query it faster downstream.
- The data must be in a fault-tolerance manner

Batch processing: Advantages

- Batch processing is simpler in the majority of use cases.



understanding



troubleshoot



application building

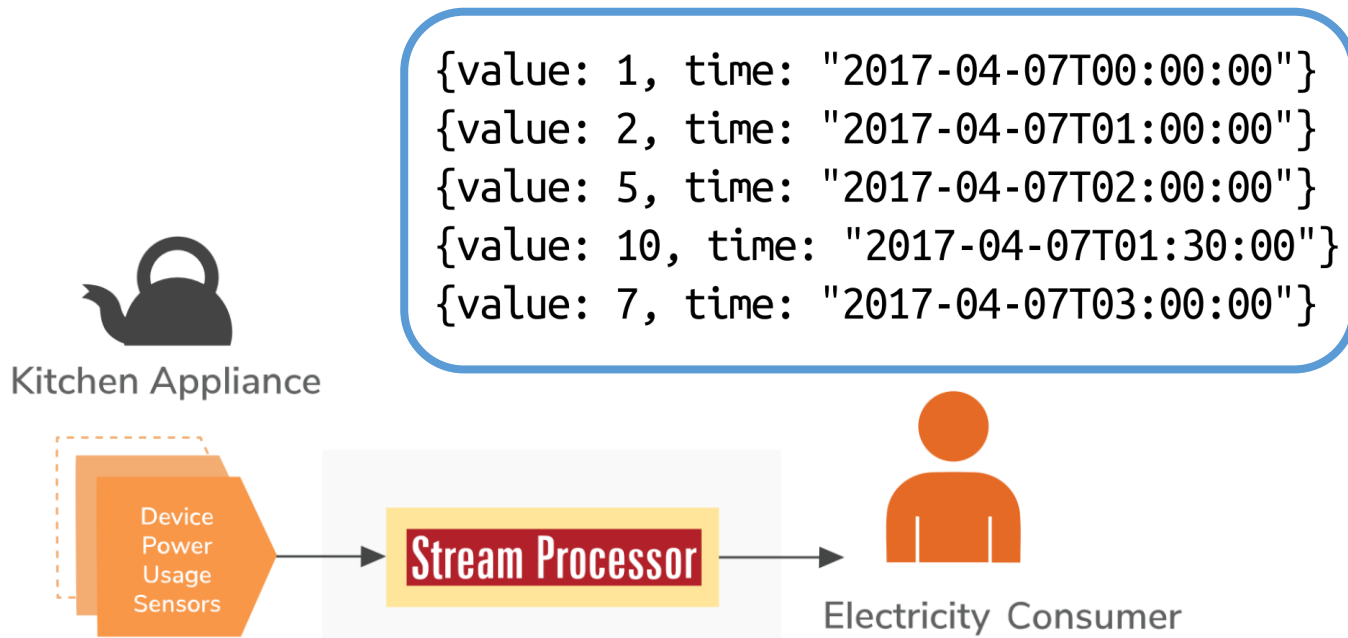
- It allows for vastly higher data processing throughput than many streaming systems.

Stream processing: Advantages

- **Lower latency:** the application needs to respond quickly (on a timescale of minutes, seconds, or milliseconds)
 - A streaming system keeps state in memory to get good performance.
 - Many decision making and alerting use cases fall into this camp.
- **More efficient in updating a result** than repeated batch jobs: the computation is automatically incrementalized
 - Case study: compute web traffic statistics over the past 24 hours
 - A streaming system recalls the state from the previous computation and only count the new data, instead of scanning all the data.

Stream processing: Challenges

- Consider an application that receives input messages from a sensor, which reports values at different times.



- We want to search the stream for a certain value or pattern of values.
- Unfortunately, the input records might arrive in an out-of-order fashion due to delays and retransmissions.

Stream processing: Challenges

- **Case study:** trigger some action based on a specific sequence of values received, say, 2 then 10 then 5.
- **Batch processing:** not notably difficult, it can simply sort all the events
- **Stream processing:** more challenging, it needs to track some state across events to realize the actual order of events

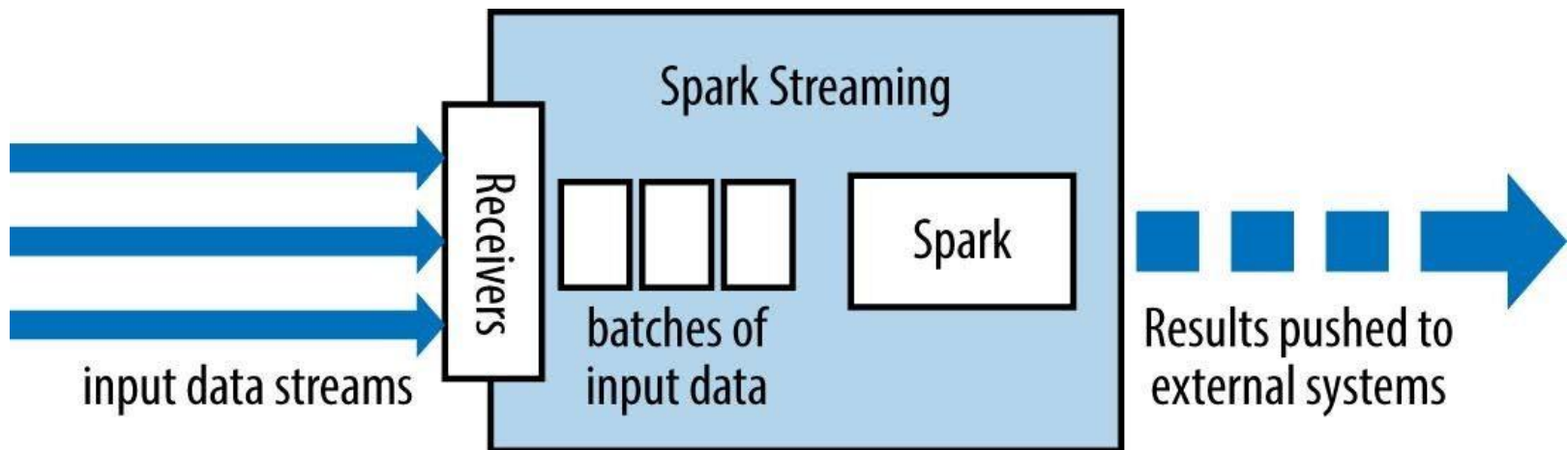
Streaming API in Spark

The history of Spark Streaming

- Spark has a long history of high-level support for streaming.
- [Spark Streaming](#) (2012) uses DStreams API, which is based on relatively low-level operations on Java/Python objects.
 - It is one of the first APIs to enable stream processing using high-level functional operators like map and reduce.
 - However, the opportunities for higher-level optimization is limited.
 - It is purely micro-batch oriented and based on processing time.
- [Structured Streaming](#) (2016, stable since v.2.2) is built on DataFrames, attaining rich optimizations and truly simpler integration with other code.
 - It offers a superset of the majority of the functionality of DStreams.
 - Better performance due to code generation and Catalyst optimizer
 - Higher-level optimizations, event time, and support continuous processing

Spark Streaming

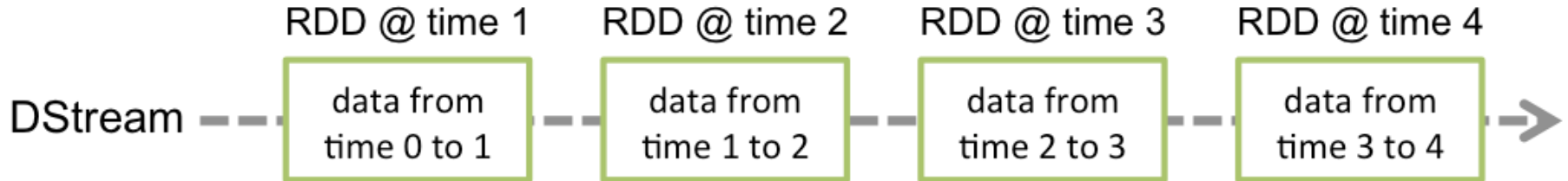
- Spark Streaming has a [micro-batch architecture](#)
 - The transmission is a series of data batches, which are created at regular time intervals.
 - The batch interval is usually between 500ms and several seconds.



[Credit](#)

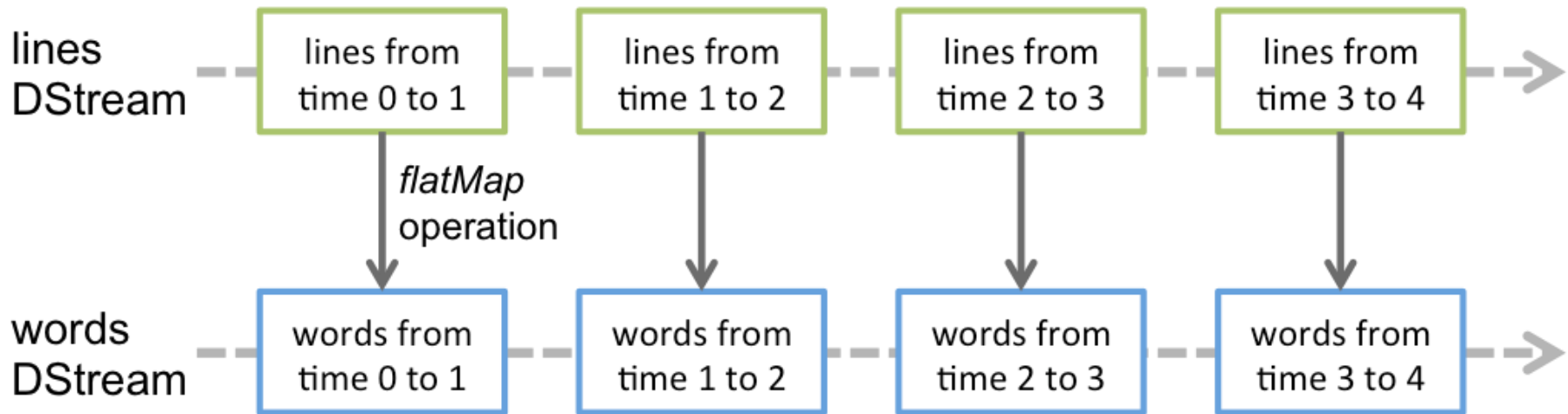
Spark Streaming: DStream API

- **DStream** is the **basic abstraction in Spark Streaming**, which represents a continuous flow of data.
 - The data flow is either the input stream received from a source or the processed data stream generated by transforming the input stream.



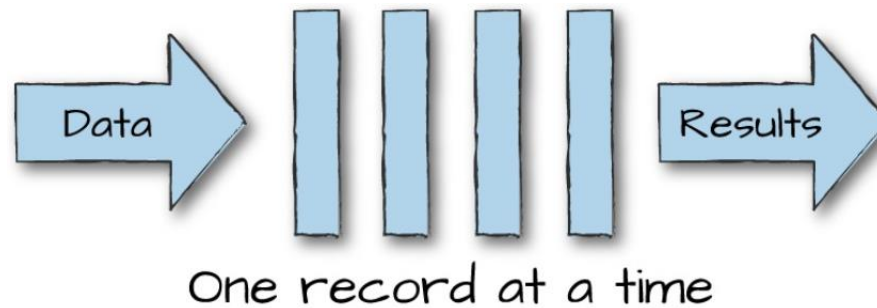
Spark Streaming: DStream API

- Any operation taken in a DStream is translated into those in the underlying RDDs.

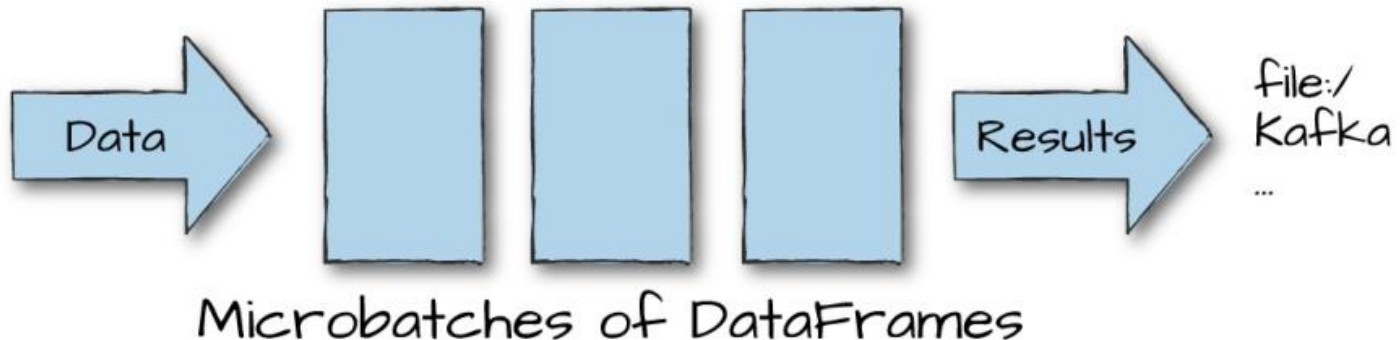


Structured Streaming

- **Continuous execution mode** has been an option for users from Spark 2.3 (however, this function is **quite limited**).



Continuous processing vs. Micro-batch execution



Structured Streaming: Continuous mode

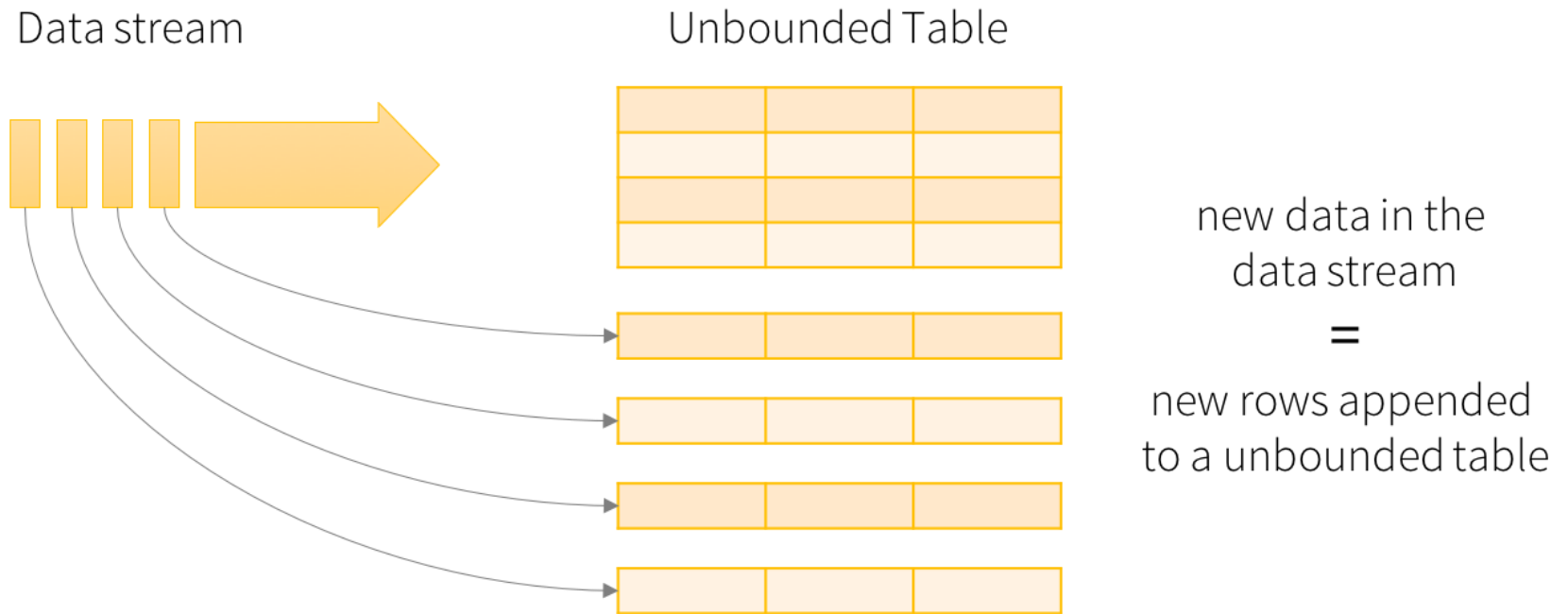
- **Continuous processing:** one record at a time
 - Each node in the system is continually listening to messages from other nodes and outputting new updates to its child nodes.
- E.g., an application implements a map-reduce computation over several input streams.
 - Each of the nodes implementing map would read records one by one from an input source, compute its function on them, and send them to the appropriate reducer.
 - The reducer then updates its state whenever it gets a new record.
- **Lowest possible latency, lower maximum throughput**

Structured Streaming: Micro-batch mode

- **Micro-batch processing:** the system accumulates small data batches (e.g., in 500 ms) then process each batch.
 - Each batch is processed similarly to the execution of a batch job.
- **High throughput per node:** leverage same optimizations as batch systems (e.g., vectorized processing), and no any extra per-record overhead incurred.
- **High latency** due to waiting to accumulate a micro-batch
- In practice, a fairly large streaming application **needs to distribute its computation to prioritize throughput.**

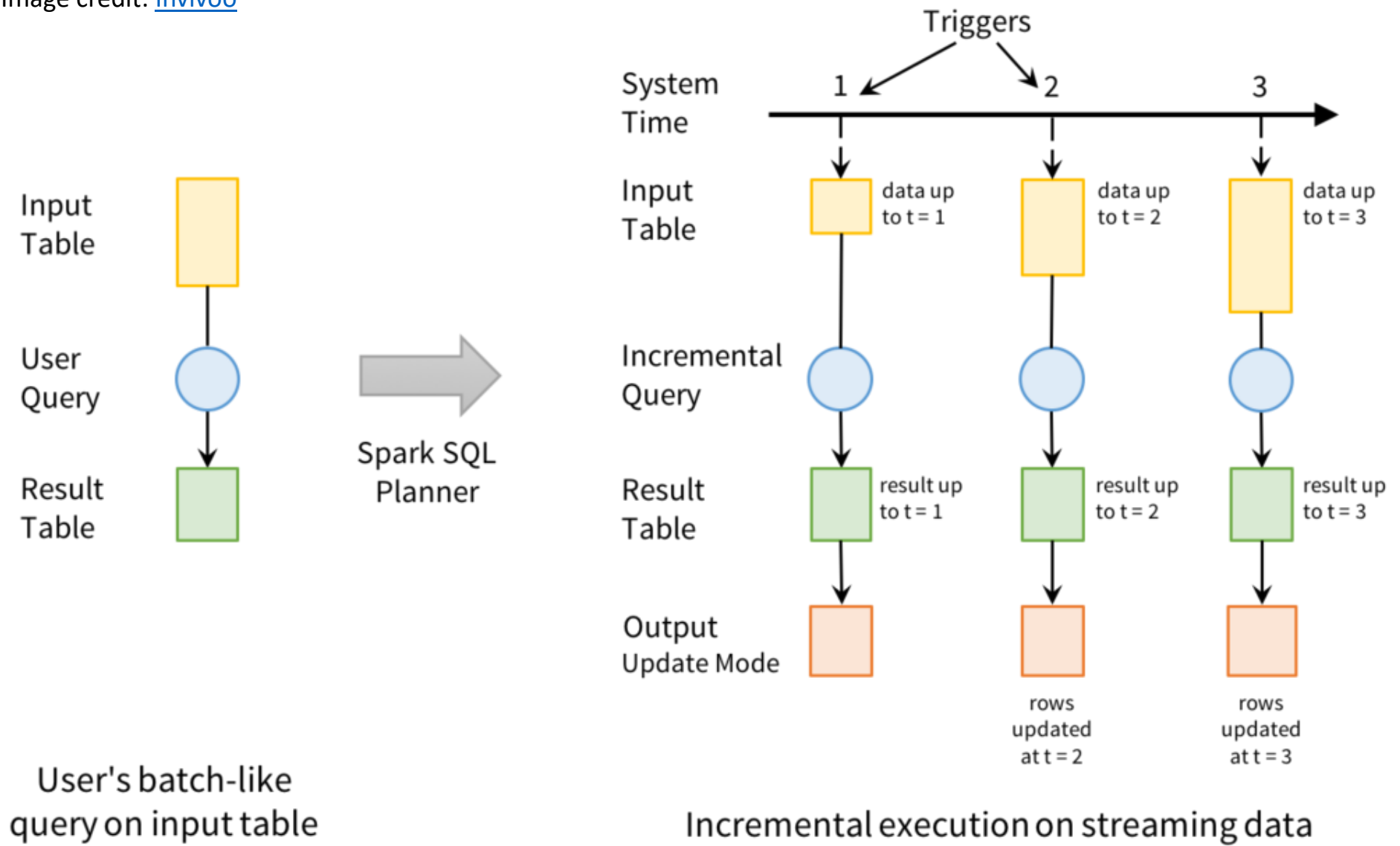
Streaming processing: Input data

- Consider the input data stream as the “**Input Table**”.
- Every data item that is arriving on the stream is like a new row being appended to the **Input Table**.



Streaming processing: Query

- For every trigger interval (e.g., every 1 second), new rows get appended to the **Input Table**.
- A query on the input will generate the **Result Table** or update it.
- Whenever the result table gets updated, we would want to write the changed result rows to an external sink.



Structured Streaming Processing Model

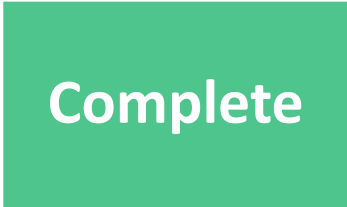
Users express queries using a batch API; Spark incrementalizes them to run on streams

Stream processing: Output data

- We usually want to write the output incrementally for each time the Result table is updated.
- Structured Streaming provides three output modes.

A blue rectangular box with the word "Append" in white text.

Append

A green rectangular box with the word "Complete" in white text.

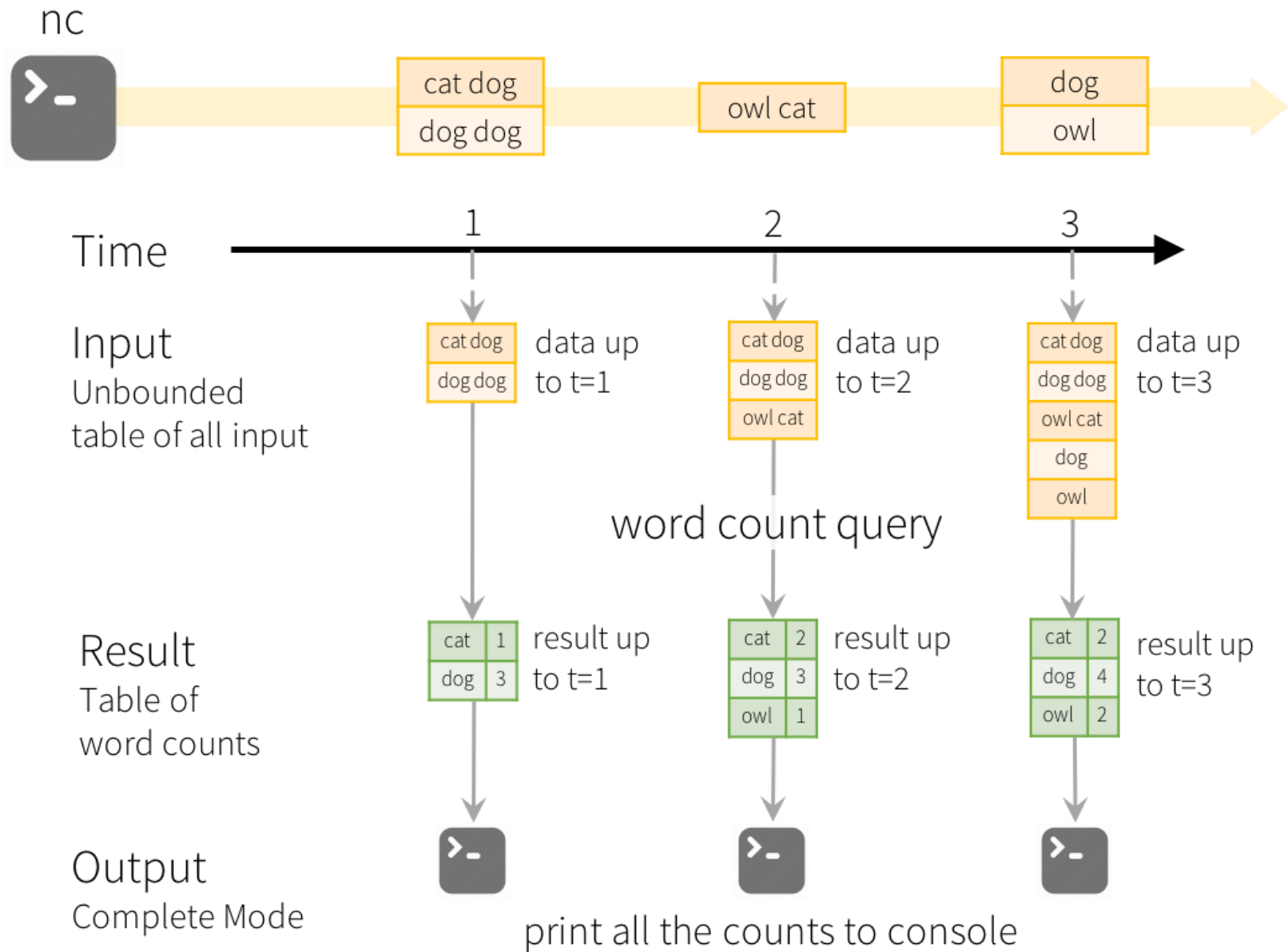
Complete

An olive green rectangular box with the word "Update" in white text.

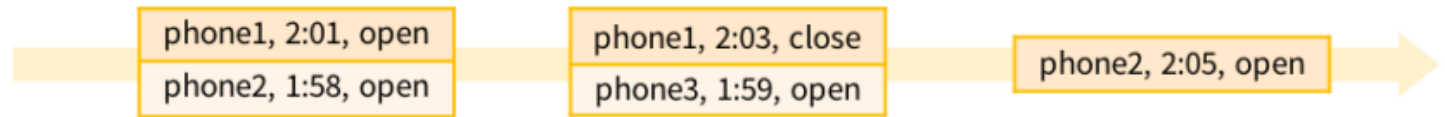
Update

Stream processing: Output modes

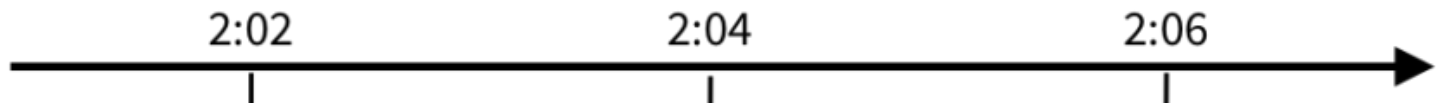
- **Append:** Only the new rows appended to the result table since the last trigger will be written to the external storage.
 - This is applicable only on queries where existing rows in the result table cannot change (e.g. a map on an input stream).
- **Complete:** The entire updated result table will be written to external storage.
- **Update:** Only the rows that were updated in the result table since the last trigger will be changed in the external storage.
 - This mode works for output sinks that can be updated in place, such as a MySQL table.



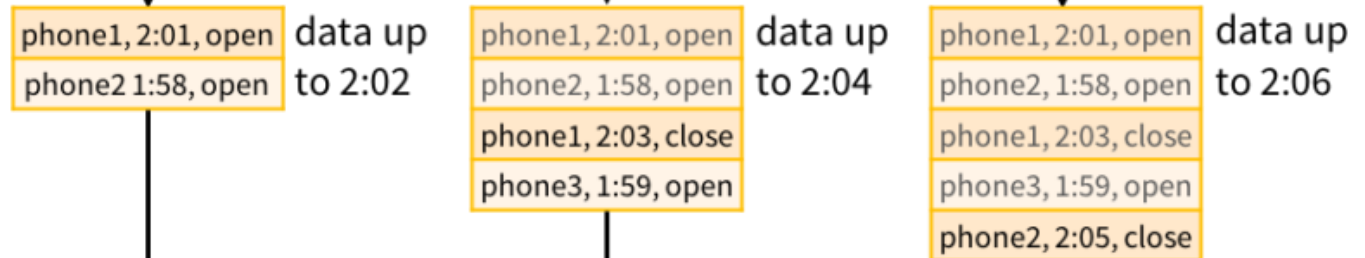
Arriving
Records



System
Time



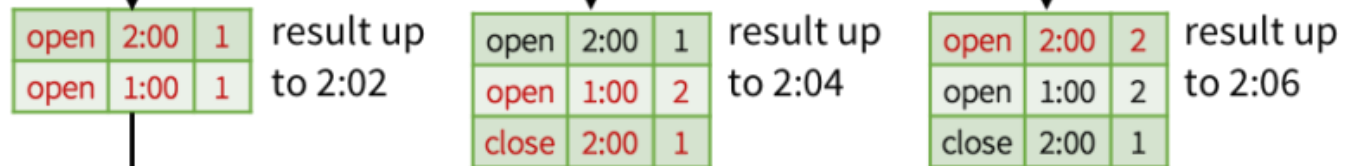
Input Table



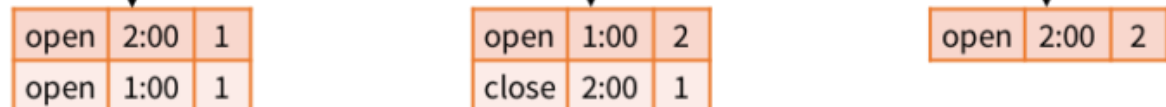
Query



Result Table

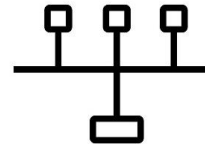
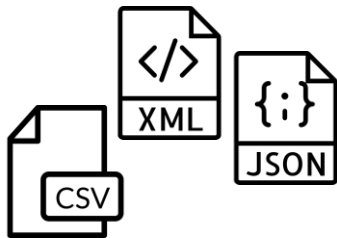


Output
Update Mode



Streaming processing: Read stream

- `DataStreamReader` can read data streams as a **Streaming DataFrames**.
- Spark generally supports several types of input source.



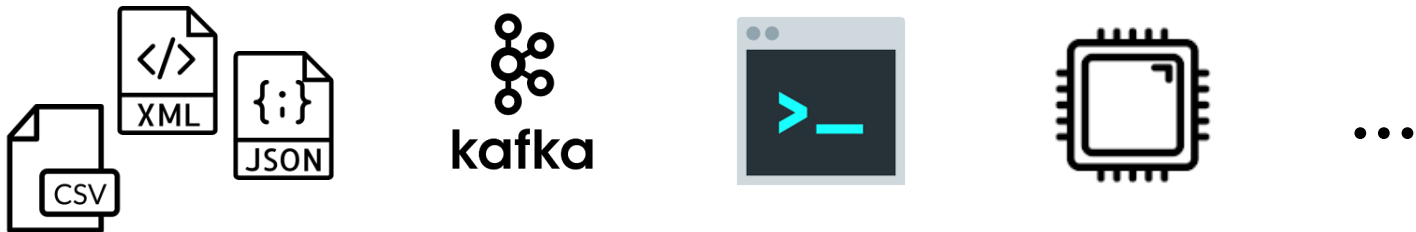
...

- For example, read data from localhost:50050

```
stream = spark.readStream.format("socket") \
    .option("host", "localhost") \
    .option("port", 50050) \
    .load()
```

Streaming processing: Write stream

- `DataStreamWriter` can save the content of the Streaming `DataFrame` into external storage.
- Spark generally supports several types of `outputSink`.



- For example, write an (aggregated) data stream to console.

```
query = processedDF.writeStream\  
    .outputMode("complete")\  
    .format("console") \  
    .start()  
query.awaitTermination(30) # in seconds
```

Streaming word count on Ncat

```
# Creating a SparkSession in Python
from pyspark.sql import SparkSession
spark = SparkSession.builder.master("local")\
    .appName("Spark Streaming Demonstration")\
    .config("spark.some.config.option", "some-value")\
    .getOrCreate()
# keep the size of shuffles small
spark.conf.set("spark.sql.shuffle.partitions", "2")
```

Streaming word count on Ncat

```
from pyspark.sql.functions import explode
from pyspark.sql.functions import split
# Create DataFrame representing the stream of input lines
# from connection to localhost:50050
lines = spark \
    .readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 50050) \
    .load()
# Split the lines into words
words = lines.select(
    explode(
        split(lines.value, " ")
    ).alias("word")
)
# Generate running word count
wordCounts = words.groupBy("word").count()
```

Streaming word count on Ncat

```
# Initialize ncat first  
# ncat server: ncat -l -p 50050  
# Start running the query that prints  
# the running counts to the console  
query = wordCounts\  
    .writeStream\  
    .outputMode("complete")\  
    .format("console") \  
    .start()  
query.awaitTermination(30)
```

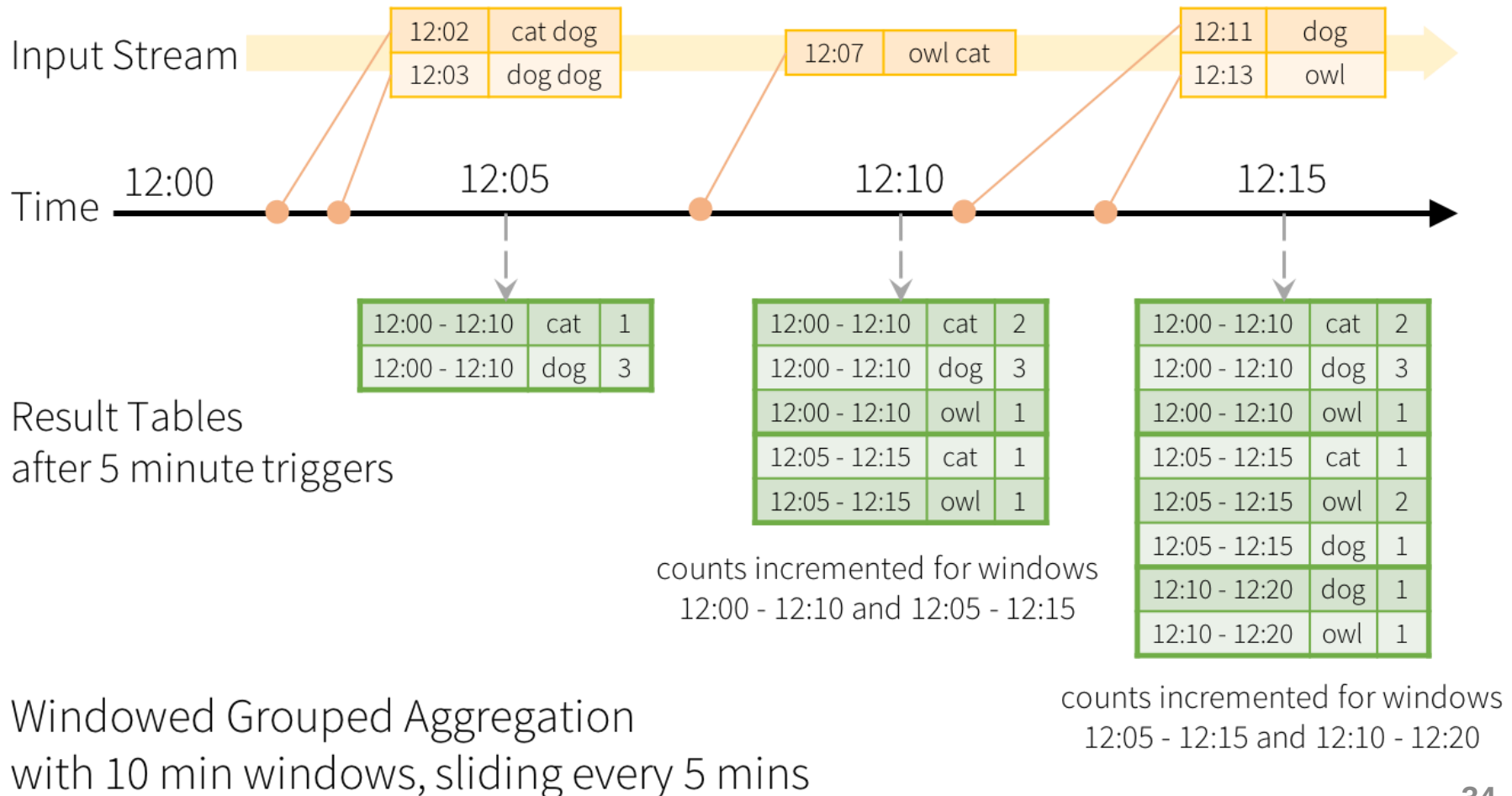
Window-based aggregations

- Imagine the stream now contains lines and timestamp.
- We want to count words within 10-minute windows, updating every 5 minutes (e.g., 12:00 - 12:10, 12:05 - 12:15, etc.)

```
words = ... # streaming DataFrame of schema { timestamp: Timestamp, word: String }  
  
# Group the data by window and word and compute the count of each group  
windowedCounts = words.groupBy(  
    window(words.timestamp, "10 minutes", "5 minutes"),  
    words.word  
)  
.count()
```

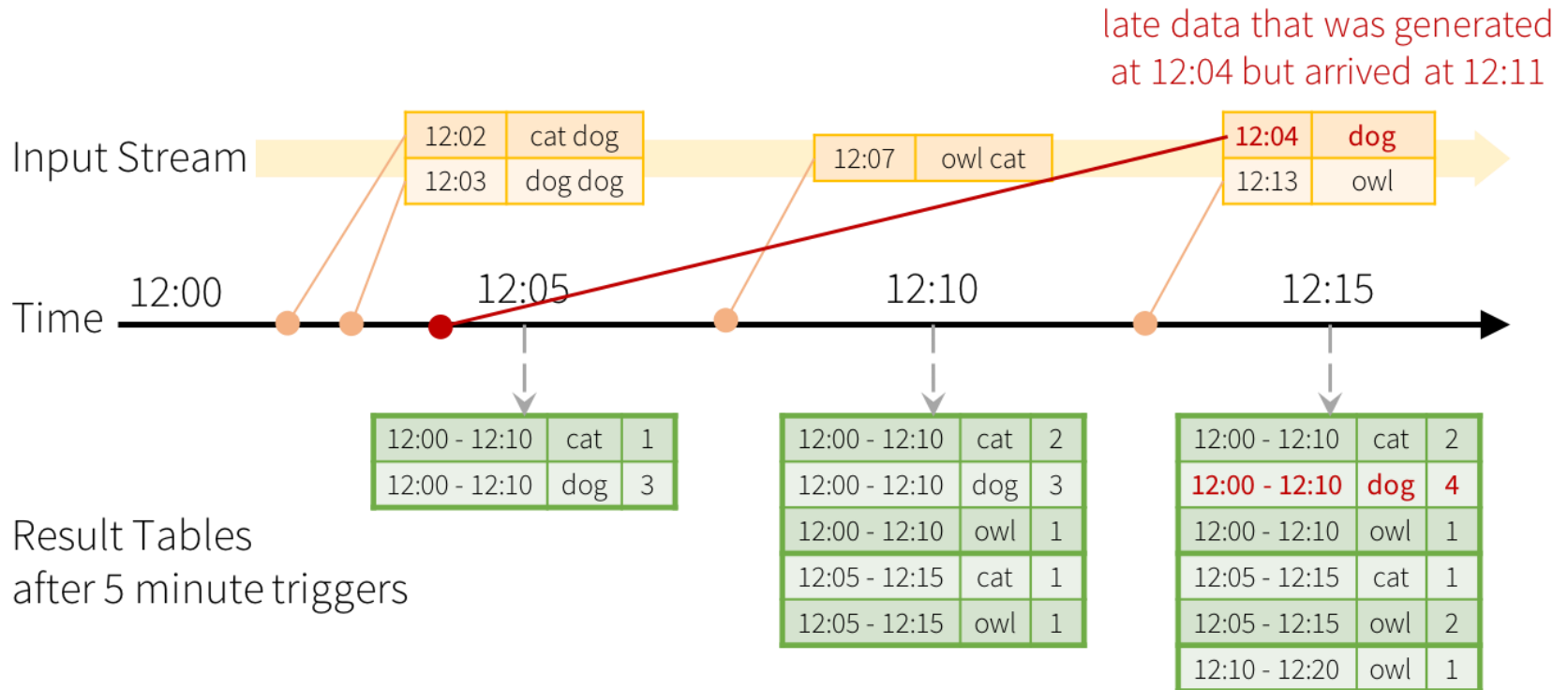
Consider a word that was received at 12:07.

This word should increment the counts corresponding to two windows 12:00 - 12:10 and 12:05 - 12:15. So the counts will be indexed by both, the grouping key (i.e. the word) and the window (can be calculated from the event-time).



Handle late data with watermarking

- Now consider what happens if one event arrives late to the application.



The application should use the time 12:04 instead of 12:11 to update the older counts for the window 12:00 - 12:10.

counts incremented only for window 12:00 - 12:10

Refer to [Structure Streaming Programming Guide](#)

Types of time windows

- **Tumbling windows** are a series of non-overlapping, fixed-sized, and contiguous time intervals.
 - An input can only be bound to a single window.
- **Sliding windows** are also fixed-sized”, but these windows can overlap.
 - An input can be bound to the multiple windows.
- Tumbling and sliding window use the `window` function.

Types of time windows

- A **session window** has a dynamic size of the window length, depending on the inputs.
 - It starts with an input, and expands itself if following input has been received within gap duration.
 - For static gap duration, a session window closes when there's no input received within gap duration after receiving the latest input.
- Session window uses `session_window` function.

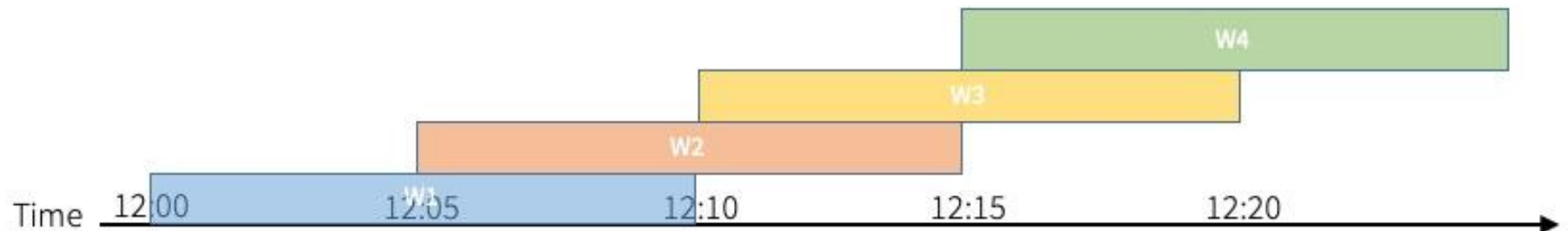
```
words = ... # streaming DataFrame of schema { timestamp: Timestamp, word: String }  
  
# Group the data by window and word and compute the count of each group  
windowedCounts = words.groupBy(  
    window(words.timestamp, "10 minutes", "5 minutes"),  
    words.word  
)  
.count()
```

Types of time windows

Tumbling Windows (5 mins)



Sliding Windows (10 mins, slide 5 mins)



Session Windows (gap duration 5 mins)



...the end.

